

Q1. Three categories of Java exceptions

Java exceptions are divided into checked exceptions, unchecked exceptions, and errors. Checked exceptions must be handled at compile time, unchecked exceptions occur at runtime, and errors represent serious system-level failures.

Q2. try-catch-finally execution order

The try block executes first, followed by the catch block if an exception occurs, and the finally block always executes. A return in finally overrides any previous return, which is why it should be avoided.

Q3. Catch order rule

Exceptions must be caught from the most specific to the most general. Violating this rule results in unreachable catch blocks and compilation errors.

Q4. throw vs throws

'throw' explicitly throws an exception instance, while 'throws' declares potential exceptions in a method signature. They are often used together when propagating exceptions.

Q5. try-with-resources

Try-with-resources automatically closes resources that implement `AutoCloseable`. Multiple resources are closed in reverse order of their creation.

Q6. Custom exception design

Extend `Exception` for recoverable conditions and `RuntimeException` for programming errors. The decision depends on whether the caller must handle the exception.

Q7. Enum characteristics

Each enum constant is an instance created using a private constructor. Enums ensure type safety and a fixed set of values.

Q8. Enum template pattern

The pattern consists of an interface, enum, exception class, and service layer. Using an interface allows flexibility across multiple enums.

Q9. List vs Set vs Queue

List allows duplicates and preserves order, Set enforces uniqueness, and Queue processes elements in FIFO order.

Q10. HashMap vs Hashtable

Hashtable is synchronized and outdated, while HashMap is unsynchronized. `ConcurrentHashMap` is the preferred modern alternative for thread safety.